Rafael Barbosa

_Desenvolvimento de Software e Operações de TI – DevOps

UNIDADE IV



Sumário

1. *Pipeline* CI/CD 3

2. Técnicas de *deployment* 11

3. AWS CodeCommit 14

4. AWS CodePipeline 40

5. AWS CodeBuild 61

1

Pipeline CI/CD

Continuous Integration

≠

Continuous Delivery

≠

Continuous Deployment

Git flow

Figura – Git flow



Fonte: elaborado pelo autor.

Automatização

Integração contínua (*Continuous integration*)

A integração contínua (CI) é uma prática de desenvolvimento originada da *Extreme Programming* que incentiva os desenvolvedores a integrarem seu código frequentemente em uma linha principal de desenvolvimento.

Figura – Fluxo do processo de integração contínua (CI)



Fonte: elaborada pelo autor.

A implementação da integração contínua oferece vários benefícios:

- Problemas de *merge*, isto é, conflitos de *merge* infinitos ao mesclar o código na linha principal, são evitados porque o código deve ser integrado à linha principal sempre que possível (pelo menos uma vez por dia).
- Permite detectar erros rapidamente, pois testes automatizados são executados a cada confirmação que verifica o repositório

remoto. Assim, menos *bugs* chegam à produção.

- Quando um erro é detectado, é muito mais fácil corrigir, porque a quantidade de código envolvida em cada confirmação é muito menor.
- Favorece a comunicação entre os membros de equipe quando ocorrem conflitos de mesclagem. As pessoas podem se sentar juntas e entender como seus códigos estão realmente influenciando o código do outro.
- Criar uma *release* torna-se mais fácil, pois todos os problemas de integração já foram resolvidos.

No entanto, a integração continua tem um custo. Para implementá-la, a equipe de desenvolvimento deve adotar as seguintes práticas:

- as tarefas devem ser divididas o mínimo possível no estágio de planejamento porque o código deve ser desenvolvido (incluindo testes automatizados) e integrado à linha principal sempre que possível, pelo menos uma vez por dia.

- Testes automatizados devem estar presentes para cada novo recurso, melhoria ou correção de *bug*. Eles garantem que uma mudança, mesmo que pequena, não quebre nenhuma outra parte do sistema.

Entrega contínua (*Continuous delivery*)

> "Entrega contínua é a capacidade de obter alterações de todos os tipos; incluindo novos recursos, alterações de configuração, correções de *bugs* e experimentos na produção, ou nas mãos dos usuários, com segurança e rapidez, de maneira sustentável."
>
> (HUMBLE, Jez; FARLEY, David. **Continuous Delivery**: Reliable Software Releases through Build, Test and Deployment Automation. Addison-Wesley, 2010).

A figura ilustra o fluxo da entrega contínua (*continuous delivery*). O processo inicia-se com a confirmação de código no repositório (*commit*). Em seguida, ocorre a compilação (*build*) da aplicação e a execução de testes automatizados. Após a validação das etapas de qualidade, um artefato é gerado e disponibilizado para implantação. O objetivo é manter a aplicação permanentemente pronta para implantação em

produção, permitindo que a liberação ocorra de forma rápida, segura e controlada sempre que necessário.

Figura – Fluxo de entrega contínua



Fonte: elaborada pelo autor.

Quando uma equipe implementa efetivamente a entrega contínua, isso significa que a linha principal está sempre em um estado implantável e que é possível implantá-la em produção a qualquer momento com o simples acionamento de um botão. Isso só é possível porque, quando esse botão é acionado, um *pipeline* automatizado (um conjunto de etapas automatizadas pelas quais o código passa a ser implantado em produção) é executado. O elemento-chave para alcançar a entrega contínua é a automação!

Bem, se você pode implantar a produção a qualquer momento, a pergunta é: quando é a hora certa? Naturalmente, a resposta pode variar dependendo dos requisitos do seu negócio. No entanto, se você deseja obter os benefícios da entrega contínua, o ideal é implantar em produção o mais cedo possível, para ter certeza

de liberar pequenos lotes fáceis de usar e solucionar problemas em caso de problemas.

Continuous deployment

A implantação contínua (*continuous deployment*) é um passo além da entrega contínua. Nesse modelo, toda alteração aprovada nos testes automatizados é implantada automaticamente em produção, sem necessidade de aprovação manual.

A figura apresenta o fluxo de implantação contínua. Após a confirmação do código no repositório, ocorre a compilação da aplicação e a execução dos testes automatizados. Quando todas as validações são aprovadas, a aplicação segue para implantação. Diferentemente da Entrega Contínua, na Implantação Contínua não existe uma etapa de aprovação manual antes da produção. Assim, toda alteração aprovada pelos testes é disponibilizada automaticamente para os usuários finais.

Figura – Fluxo de integração contínua

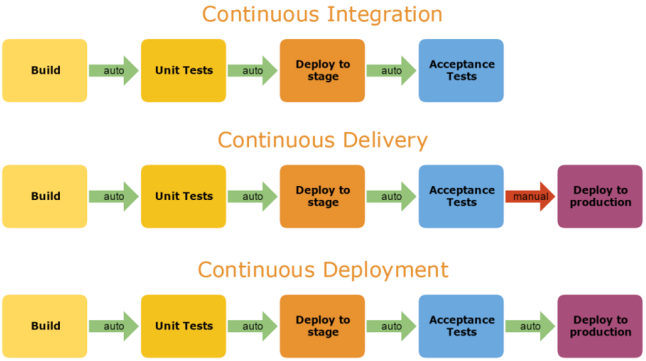


Fonte: elaborada pelo autor.

Como na implantação contínua, toda mudança que passa na suíte de testes é implantada na produção, ela depende muito dos *feature flags* para liberar recursos para os usuários finais.



Figura – Automatização



Fonte: elaborado pelo autor.

A figura apresenta uma comparação entre os três níveis de automação do ciclo de entrega de *software*:

- **Integração contínua (*continuous integration*)**

O código é integrado frequentemente ao repositório principal. Compilações e testes

automatizados são executados regularmente. O foco está na qualidade e integração do código.

- **Entrega contínua (*continuous delivery*)**

Inclui tudo o que existe na integração contínua. Mantém a aplicação permanentemente pronta para implantação.

A implantação em produção ainda depende de uma decisão ou aprovação humana.

- **Implantação contínua (*continuous deployment*)**

Inclui todas as práticas de entrega contínua. Toda alteração aprovada pelos testes automatizados é implantada automaticamente em produção. Não há necessidade de aprovação manual antes da publicação.

Técnicas de *deployment*

Rolling updates

Figura - Rolling updates

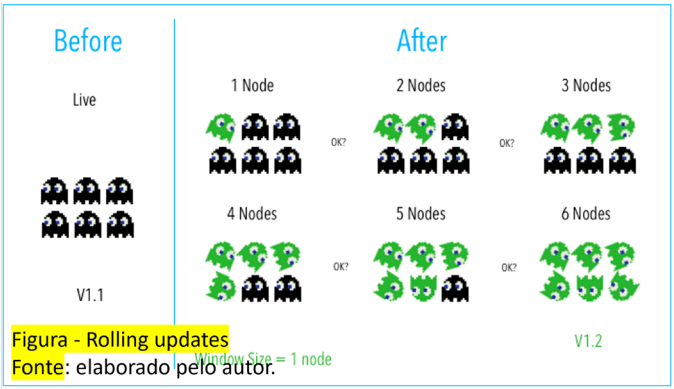


Figura - Rolling updates
Fonte: elaborado pelo autor.

Fonte: elaborado pelo autor.

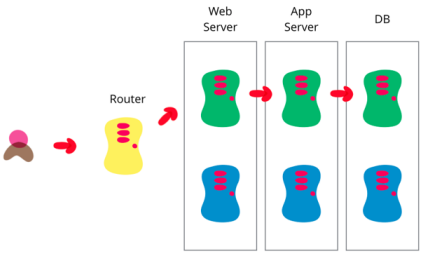
Chaveamento ocorre aos poucos.

Atualiza-se um (ou mais) servidor por vez.

Blue-green

Blue-green deployments

Figura - Blue-green deployments



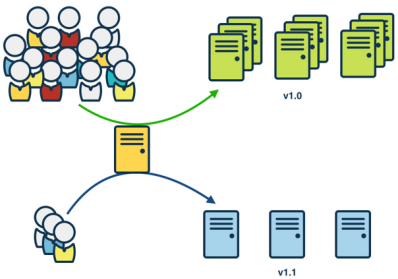
Fonte: elaborado pelo autor.

Stacks inteiras em produção simultaneamente. Chaveamento em poucos segundos.

Canary

Canary deployments

Figura – Canary deployments



Fonte: elaborado pelo autor

Stacks inteiras em produção simultaneamente.

Chaveamento aos poucos de acordo com os resultados.

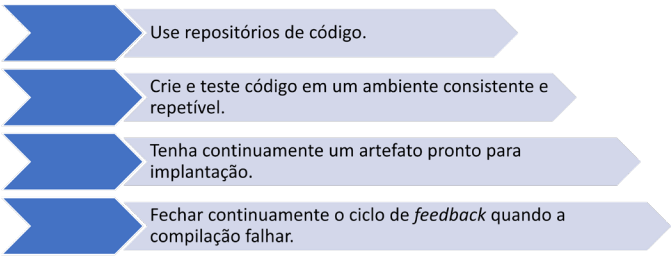
3

AWS CodeCommit

Integração contínua

Prática de mesclar regularmente as alterações de código em um repositório central e executar a construção e o teste automatizados.

Figura – Integração contínua



Fonte: elaborada pelo autor.

O compartilhamento frequente de código é um aspecto fundamental da integração contínua. Nas práticas de desenvolvimento contínuo, diversas pessoas trabalham no código. É fundamental que todos usem a versão funcional mais recente para

seus esforços. Os repositórios de código mantêm diferentes versões do código e também tornam o código acessível à equipe. Você verifica o código do repositório, faz suas alterações ou escreve um novo código em sua cópia local, compila e testa seu código e, em seguida, envia frequentemente seu código de volta ao repositório.

Para alcançar a integração contínua, é essencial:

- manter a integridade do código no repositório;
- manter a versão funcional mais recente no repositório.

Isso foi projetado de forma que todos que fazem *check-out* do código do repositório usem a compilação mais recente e comecem com uma cópia de trabalho (não uma compilação quebrada). Para habilitar isso, você deve executar a fase de construção sempre que confirmar o código no repositório. Diversas ferramentas no mercado já automatizam esse processo acionando automaticamente uma fase de construção após o *commit* do código. Todas as etapas da fase de construção (construir, empacotar e registrar no repositório de pacotes) são executadas automaticamente.

A integração contínua refere-se aos estágios de construção e teste unitário do processo de lançamento do *software*. Cada revisão confirmada aciona uma construção e um teste automatizados.

Como o teste de unidade avalia unidades individuais do código do aplicativo, ele é útil para identificar e isolar problemas rapidamente. Como parte de um processo automatizado de CI/CD, ele também pode ser usado para evitar que códigos incorretos sejam implantados na produção.

Vantagens: melhora a produtividade do desenvolvedor, encontra e resolve *bugs* com mais rapidez e entrega atualizações com mais rapidez.

O que é controle de origem?

Prática de rastrear e gerenciar alterações no código

- **Fundamentos**:
  - Rastreie alterações, visualize o histórico de revisões, reverta para versões anteriores, colabore, isole e identifique alterações rapidamente.

- **Exemplo de sistema de gerenciamento de código-fonte:**
  - Git, Mercurial, Apache, subversion.
- **Ferramentas:**
  - AWS CodeCommit, GitHub e outros.

Um aspecto importante do CI é o código. Para garantir que você tenha a mais alta qualidade do código, é importante ter controle de origem. O controle de origem é a prática de rastrear e gerenciar alterações no código. Os sistemas de gerenciamento de controle de origem (SCM) fornecem um histórico do desenvolvimento do código e ajudam a resolver conflitos ao mesclar contribuições de diversas fontes.

Noções básicas de controle de origem

Esteja você escrevendo um aplicativo simples sozinho ou colaborando em um grande projeto de desenvolvimento de *software* como parte de uma equipe, o controle de origem é um componente vital do processo de desenvolvimento. Com o gerenciamento de código-fonte, você pode acompanhar as

alterações no código, ver um histórico de revisões do seu código e reverter para versões anteriores de um projeto quando necessário.

Ao usar sistemas de gerenciamento de código-fonte, você pode:

- colaborar no código com sua equipe;
- isolar seu trabalho até que esteja pronto;
- solucionar problemas rapidamente identificando quem fez as alterações e quais foram.

Os sistemas de gerenciamento de código-fonte ajudam a agilizar o processo de desenvolvimento e fornecem uma fonte centralizada para todo o seu código.

O que é Git?

Git é um sistema de gerenciamento de código-fonte distribuído de código aberto. Use o Git para criar uma cópia do seu repositório conhecida como ramificação (*branch)*. Usando essa ramificação, você pode trabalhar em seu código independentemente da versão estável de sua base de código. Quando suas alterações

estiverem prontas, você poderá armazená-las como um conjunto de diferenças, conhecido como *commit*. Você pode obter *commits* de outros colaboradores para o seu repositório, enviar seus *commits* para outros e mesclar seus *commits* de volta na versão principal do repositório.

Mercurial é outra ferramenta de gerenciamento de controle de origem distribuído.

Subversion é um sistema de controle de versão de código aberto que é mantido pela Apache Software Foundation.

Controle de origem com AWS

AWS CodeCommit é um sistema gerenciado de controle de origem que hospeda repositórios Git e funciona com todas as ferramentas baseadas em Git. O AWS CodeCommit armazena código, binários e metadados de forma redundante e com alta disponibilidade. Você poderá colaborar com equipes locais e remotas para editar, comparar, sincronizar e revisar seu código.

Como o AWS CodeCommit é executado na Nuvem AWS, você não precisa mais se preocupar

com hospedagem, escalabilidade ou manutenção de sua própria infraestrutura de controle de código-fonte. O CodeCommit criptografa automaticamente seus arquivos e se integra ao AWS Identity and Access Management (IAM), permitindo atribuir permissões específicas do usuário aos seus repositórios. Isso garante que seu código permaneça seguro e que você possa colaborar em projetos com toda a sua equipe de maneira segura.

Mais informações (disponíveis em: https://aws.amazon.com/devops/source-control/).

Serviço: AWS CodeCommit



AWS CodeCommit

- Serviço de controle de origem totalmente gerenciado que hospeda repositórios seguros baseados em Git.
- Apoiado pelo Amazon S3.
- Escala com facilidade.
- Armazena qualquer coisa, a qualquer hora.

AWS CodeCommit também se integra a muitos outros serviços da AWS, como AWS IAM, Amazon CloudWatch e AWS CloudTrail. Embora a AWS CodeCommit possa armazenar muitas coisas, é importante conhecer e saber quais limites podem ser alterados.

Mais informações sobre limites com AWS CodeCommit (disponíveis em: https://docs.aws.amazon.com/codecommit/latest/userguide/limits.html).

Configuração do AWS CodeCommit

1. Crie identidade e política do IAM para acessar o AWS CodeCommit.
2. Conceda permissões anexando políticas de acesso ao usuário do IAM.
3. Instale o Git.
4. Instale a AWS CLI.
5. Configure credenciais (HTTPS/SSH ou chaves existentes com o auxiliar de credenciais AWS CLI).
6. Conecte-se ao AWS CodeCommit e clone o repositório.

Melhores práticas para controlar o acesso ao AWS CodeCommit

- Crie sua própria política.
- Forneça credenciais de acesso temporário ao seu repositório:
  - normalmente feito por meio de uma conta AWS separada para IAM e contas separadas para dev/staging/prod;
  - acesso federado;
  - autenticação multifator.

Crie sua própria política para controlar o acesso ao AWS CodeCommit. As políticas podem ser aplicadas a usuários ou grupos do AWS IAM. Considere a criação de grupos, como DeveloperGroup/QAGroup, e conceda acesso adequadamente para um controle mais granular. Para criar suas próprias políticas, você pode usar as políticas gerenciadas como modelo.

Mais informações sobre como criar sua própria política (disponíveis em: http://docs.aws.amazon.com/codecommit/latest/userguide/how-to-share-repository.html#how-to-share-repo-create-policy).

Você pode usar políticas gerenciadas do AWS IAM, como AWSCodeCommitFull Access, AWSCodeCommitPowerUser e AWSCodeCommitReadOnly , mas elas podem permitir acesso a todos os repositórios em uma conta. É uma prática recomendada criar políticas gerenciadas pelo cliente específicas para um repositório.

Supondo que uma função também seja uma boa maneira de controlar o acesso ao seu repositório, você pode configurar a AWS CLI para usar uma função criando um perfil para essa função. Cenários típicos a serem considerados para fornecer credenciais de acesso temporário ao seu repositório:

- Você deseja fornecer acesso a um usuário do IAM de uma conta separada da AWS (entre contas).
- Usuários que usam federação de identidades da web, como Amazon. com, Amazon Cognito, Facebook, Google ou qualquer provedor de identidade compatível com OIDC.

Mais informações sobre o uso de identidades federadas com AWS CodeCommit (disponíveis em: https://aws.amazon.com/blogs/devops/using-federated-identities-with-aws-codecommit/).

- Usuários com MFA habilitado em suas contas.

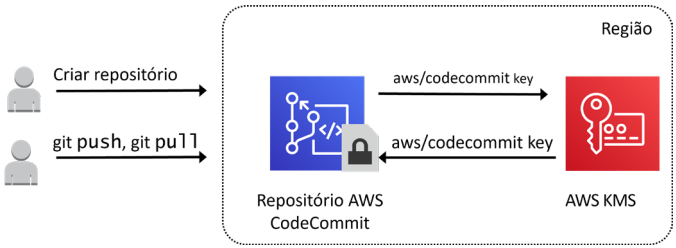
O usuário deve configurar sua máquina de desenvolvimento para usar essas credenciais para acessar os repositórios do AWS CodeCommit.

Descrição das políticas AWS IAM:

- **AWSCodeCommitFullAccess:** permite todos os acessos; sem restrições.
- **AWSCodeCommitPowerUser:** todas, mas nenhuma exclusão, de repositórios do CodeCommit ou criação/exclusão de recursos relacionados ao repositório em outros serviços da AWS, como Amazon CloudWatch Events.
- **AWSCodeCommitReadOnly:** permite usuários para extrair conteúdo; sem acesso.

Criptografia + AWS CodeCommit = Segurança

Figura – Criptografia + AWS CodeCommit = Segurança



Fonte: elaborado pelo autor.

Com o AWS CodeCommit, os dados são criptografados em trânsito e em repouso.

- Quando você envia para o AWS CodeCommit (*git push*), ele criptografa os dados e os armazena.
- Quando você extrai do AWS CodeCommit (*git pull*), ele descriptografa os dados e os envia de volta ao chamador.

O usuário deve ser autenticado do IAM com o nível de acesso adequado ao AWS CodeCommit.

Na primeira vez que você cria um repositório do AWS CodeCommit em uma nova região na sua conta da AWS, o AWS CodeCommit cria uma chave gerenciada pela AWS. Essa chave é

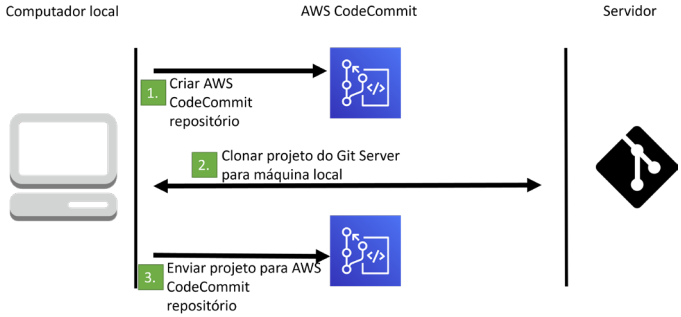
criada e armazenada em sua conta AWS. AWS O CodeCommit usa essa chave gerenciada pela AWS para criptografar e descriptografar os dados nesse e em todos os outros repositórios do AWS CodeCommit nessa região em sua conta da AWS.

Os dados são criptografados em trânsito porque são transmitidos por meio de conexões de rede criptografadas usando protocolos HTTPS ou SSH. O contexto de criptografia consiste em informações adicionais que o AWS Key Management Service (AWS KMS) usa para verificar a integridade dos dados. O AWS KMS usa o ID do repositório do AWS CodeCommit para o contexto de criptografia. Para encontrar o ID do repositório, execute o comando get-repository.

Para obter mais informações, pesquise o ID do repositório do AWS CodeCommit nos *logs* do AWS CloudTrail. Ao fazer isso, você pode entender quais operações de criptografia foram realizadas em uma chave específica no AWS KMS para criptografar ou descriptografar dados no AWS CodeCommit repositório.

Migrar um repositório Git para AWS CodeCommit

Figura – Migrar um repositório Git para AWS CodeCommit



Fonte: elaborado pelo autor.

O diagrama mostra as principais etapas na migração de um repositório Git para o AWS CodeCommit.

1. Em um computador local, um usuário com acesso apropriado ao AWS CodeCommit cria um repositório do AWS CodeCommit.
2. No servidor Git, o projeto git é clonado no computador local.
3. No computador local, envie o clone para o repositório do AWS CodeCommit.

Se o seu repositório atual for muito grande para ser migrado, considere migrar para o AWS CodeCommit de forma incremental. Fazer isso reduz o risco de falha de um processo git push devido a problemas de rede.

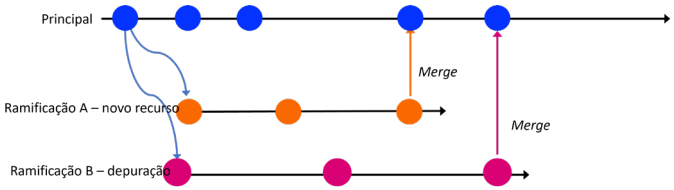
Aqui estão alguns comandos/opções úteis a serem considerados ao enviar ou capturar as referências:

- ***git clone:*** clonar um repositório em um novo diretório;
- ***git push:*** atualiza referências remotas junto com objetos associados;
- **all:** em vez de nomear cada referência para *push*, especifica que todas as referências em $GIT_DIR/refs/heads/ sejam enviadas;
- **mirror:** em vez de nomear cada referência para *push*, especifica que todas as referências em $GIT_DIR/refs/ (que inclui, mas não está limitado a, refs/heads/, refs/remotes/ e refs/tags/) sejam espelhadas para o repositório remoto.
- **tags:** todas as referências em $GIT_DIR/refs/tags são enviadas, além das refspecs listadas explicitamente na linha de comando.

Mais informações sobre como migrar para o AWS CodeCommit (disponíveis em: https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-migrate-repository.html e https://git-scm.com/docs).

Ramificação principal

Figura – Ramificação principal

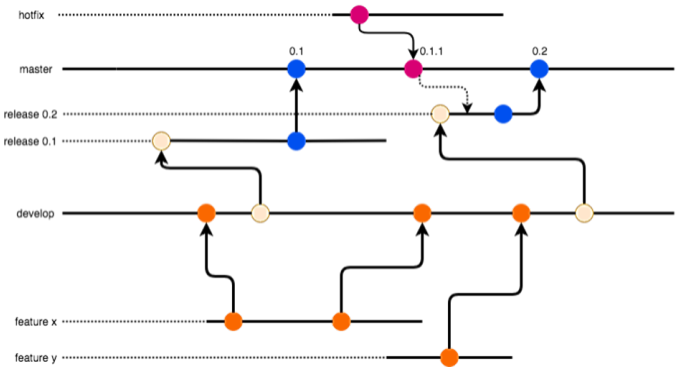


Fonte: elaborado pelo autor.

O diagrama mostra um repositório com uma ramificação principal e duas ramificações para novos recursos e depuração. A integração contínua do trabalho dessas duas filiais na linha principal é uma prática recomendada.

Trabalhando com *branchs*

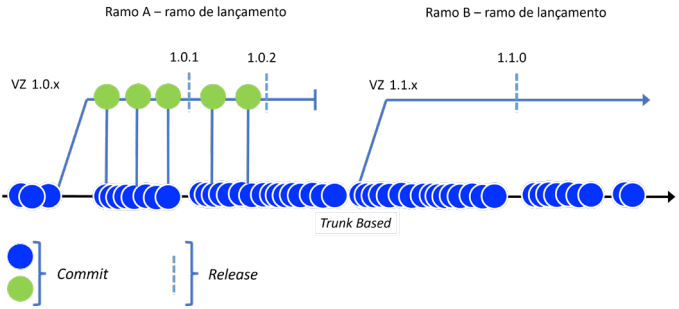
Figura – Trabalhando com branchs



Fonte: elaborado pelo autor.

Desenvolvimento baseado em tronco

Figura – Desenvolvimento baseado em tronco



Fonte: elaborado pelo autor.

O desenvolvimento baseado em tronco é outro modelo de ramificação no qual muitos desenvolvedores integram ou se comprometem com uma ramificação compartilhada conhecida como tronco. Esse *branch* geralmente está sob controle de origem, então a maioria dos *commits* acontecerá no *trunk*. As outras ramificações criadas são para lançamento e, após a criação de outra ramificação, geralmente a ramificação mais antiga é excluída. As ramificações de lançamento ficam em funcionamento por um curto período de tempo e os *commits* são controlados por indivíduos ou funções específicas na equipe. A maioria dos ajustes/correções/*commits* acontecem no tronco e são então mesclados no *branch* de lançamento.

Trabalhando com *Pull Requests*

Pull request: uma forma de os desenvolvedores colaborarem em um repositório.

- Revise e mescle alterações de código.
- Veja comparações entre *branchs* de trabalho e de destino:

- de origem = inclui o código que você deseja revisar;
- de destino = onde você mescla o código revisado.

- Atualize a *pull request* em resposta aos comentários.
- Configure notificações de repositório para alertar sobre nova *pull request*.

As solicitações *pull* são uma maneira comum de os desenvolvedores notificarem e revisarem o trabalho uns dos outros antes de serem mesclados em ramificações de código comuns. Eles fornecem uma interface *web* amigável para discutir as alterações propostas antes de integrá-las ao projeto oficial. Se houver algum problema com as alterações propostas, estas poderão ser discutidas e o código-fonte ajustado para satisfazer os requisitos de codificação de uma organização.

As solicitações *pull* vão além das simples notificações do desenvolvedor, pois permitem que discussões completas sejam gerenciadas na construção do repositório, em vez de fazer com que você dependa de trilhas de *e-mail*.

Você pode criar solicitações *pull* por meio do console do AWS CodeCommit ou da AWS CLI. Primeiro, você cria uma ramificação para suas alterações de código. Isso é conhecido como ***branch* de origem** para uma solicitação *pull*. Depois de confirmar e enviar alterações por *push* para o repositório, você pode criar uma solicitação *pull* que compara o conteúdo dessa ramificação (a ramificação de origem) com a ramificação em que deseja mesclar suas alterações após o fechamento da solicitação *pull* (a ramificação de destino).

Você pode usar o console do AWS CodeCommit para revisar as alterações incluídas em uma solicitação *pull*. Você pode adicionar comentários de texto ou *markdown* à solicitação, aos arquivos e a linhas específicas de código. Você também pode responder aos comentários feitos por outros usuários. Se o seu repositório estiver configurado com notificações, você receberá *e-mails* quando os usuários responderem aos seus comentários ou quando os usuários comentarem em uma solicitação *pull*.

Você pode usar a AWS CLI para comentar uma solicitação *pull* e responder aos comentários. Para revisar as alterações, você deve usar o comando *git diff* ou uma ferramenta *diff*.

Quando estiver satisfeito com a revisão do seu código, você poderá fechar uma solicitação *pull* de várias maneiras, como segue.

- Em seu computador local, você pode usar o comando *git merge* para mesclar a ramificação de origem na ramificação de destino e, em seguida, enviar o código mesclado para a ramificação de destino. Isso fecha a solicitação *pull* automaticamente. Use o comando *git merge* para escolher a opção ou estratégia de mesclagem. Esta opção mescla as ramificações se o *status* da solicitação *pull* mostrar "Resolver conflitos". Para saber mais sobre as opções de mesclagem e mesclagem do *git*, consulte a documentação do Git.
- No console, você pode fechar uma solicitação *pull* sem mesclar o código. Talvez você queira fazer isso se quiser usar o comando *git merge* para mesclar as ramificações manualmente ou se o código

na ramificação de origem da solicitação *pull* não for o código que você deseja mesclar na ramificação de destino. Use esta opção se o código não puder ser mesclado automaticamente. Você verá uma mensagem de aviso para resolver conflitos, o que deve ser feito em seu computador local com o comando *git merge* ou uma ferramenta *diff* ou *merge*.

- No console, você poderá mesclar automaticamente a ramificação de origem à ramificação de destino, fechando a solicitação *pull* automaticamente. Você verá uma mensagem informando que a solicitação *pull* é capaz de mesclar e o botão **Mesclar** na solicitação *pull* está ativo. Quando você escolhe **Merge**, a mesclagem é executada usando a opção de mesclagem de avanço rápido.
- Se a ramificação de origem ou de destino da solicitação *pull* for excluída, o AWS CodeCommit fechará uma solicitação *pull* automaticamente.
- Na AWS CLI, você pode atualizar o *status* de uma solicitação *pull* de OPEN para CLOSED.

Isso fecha a solicitação *pull*. Você também pode usar a AWS CLI para tentar mesclar e fechar a solicitação *pull*.

Além disso, você pode querer configurar uma ramificação para que apenas alguns usuários do repositório possam enviar ou mesclar código nela. Por exemplo, talvez você queira configurar uma ramificação usada para código de produção para que apenas um subconjunto de desenvolvedores seniores possa enviar ou mesclar alterações nessa ramificação. Outros desenvolvedores ainda podem extrair código da ramificação, criar suas próprias ramificações e criar solicitações *pull*, mas não podem enviar ou mesclar alterações nela. Você pode configurar esse acesso criando uma política condicional que usa uma chave de contexto para uma ou mais ramificações no IAM.

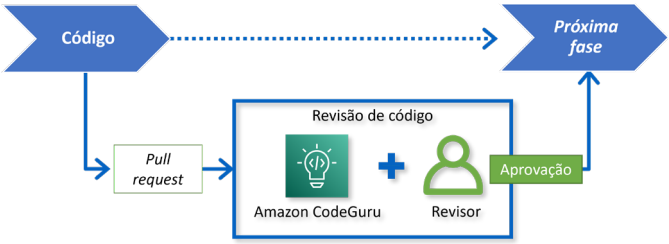
Mais informações sobre como limitar *pushes* e mesclar ramificações (disponíveis em: https://docs.aws.amazon.com/codecommit/latest/userguide/how-to-conditional-branch.html).

Revisão de código com Amazon CodeGuru Reviewer – *Pull Request*



Revisor do Amazon CodeGuru
Revisões de código integradas com recomendações acionáveis

Figura – Revisão de código com Amazon CodeGuru Reviewer – Pull Request



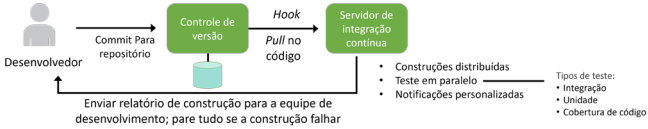
Fonte: elaborado pelo autor.

Automatize usando *Hooks* e notificações do lado do servidor

Use ganchos do lado do servidor em combinação com servidores de CI para reduzir a resistência e aumente a velocidade de suas integrações.

Use notificações e gatilhos do CodeCommit para enviar relatórios e invocar funções do Lambda.

Figura – Automatize usando Hooks e notificações do lado do servidor



Fonte: elaborado pelo autor.

Seu processo de desenvolvimento usará ferramentas que funcionem para sua equipe, como Git, CodeCommit ou sua solução customizada. Não importa qual ferramenta você use, você deve automatizar. Como exemplo, considere o seguinte. Usando ganchos de pós-recebimento (*post-receive hooks*), você pode direcionar novas ramificações para acionar testes em um servidor de integração contínua para verificar se a nova compilação está integrada adequadamente e se todas as unidades funcionam corretamente. O desenvolvedor é notificado sobre falhas de teste e só sabe que deve mesclar sua ramificação com a linha principal depois que os problemas forem corrigidos. O desenvolvedor pode construir a partir de sua ramificação, testar as alterações e obter *feedback* sobre o funcionamento de suas

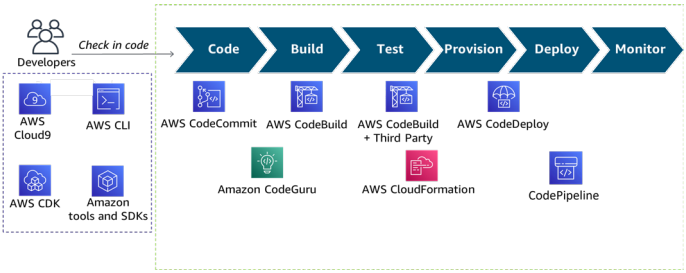
alterações antes de decidir se deve mesclar sua ramificação na linha principal.

A execução de testes unitários e de integração reduz significativamente as resistências quando esse ramo é mesclado na linha principal.

Você também pode personalizar ganchos para testar mesclagens na linha principal e bloquear quaisquer mesclagens que não sejam aprovadas. Tudo isso é mais bem realizado com um servidor CI.

Ferramentas e automação

Figura – Ferramentas e automação



Fonte: elaborado pelo autor

4

AWS CodePipeline

Serviço: AWS CodePipeline

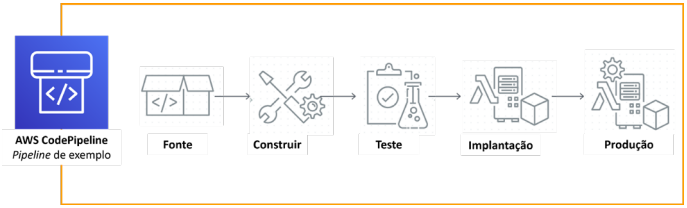


AWS CodePipeline

- Serviço de entrega contínua para atualizações de aplicativos rápidas e confiáveis.
- Coordena a construção, o teste e a implantação do seu código.
- Integra-se com ferramentas de terceiros e AWS.
- Use-o para modelar e visualizar seu processo de lançamento de *software*.

Exemplo: CodePipeline

Figura – Exemplo: CodePipeline



Fonte: elaborado pelo autor.

O AWS CodePipeline pode extrair o código-fonte do seu *pipeline* diretamente do AWS CodeCommit, GitHub, Amazon Elastic Container Registry (Amazon ECR) ou Amazon S3. Ele pode executar compilações e testes de unidade no AWS CodeBuild. O CodePipeline pode implantar suas alterações usando AWS CodeDeploy, AWS Elastic Beanstalk, Amazon ECS ou AWS Fargate.

O CodePipeline também oferece suporte a ações do AWS CloudFormation que permitem provisionar, atualizar ou excluir recursos da AWS como parte do processo de lançamento. Isso também permite que você forneça continuamente aplicativos sem servidor criados usando AWS Lambda, Amazon API Gateway e Amazon DynamoDB com o AWS Serverless Application Model (AWS SAM).

Você também pode acionar funções personalizadas definidas em código em qualquer estágio do pipeline usando a integração do CodePipeline com o AWS Lambda. Por exemplo, você pode acionar uma função Lambda que testa se sua aplicação *web* foi implantada com sucesso.

Com o CodePipeline, você pode configurar um *pipeline* que une esses serviços, além de ferramentas de desenvolvedor de terceiros e sistemas personalizados.

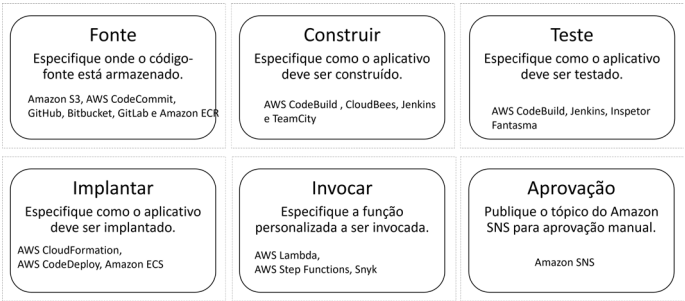
Metas de pipeline

- Implante com segurança sem impactar clientes e negócios.
- Validar/testar código:
  - teste de nível de código;
  - a integração com dependências diretas está funcionando corretamente;
  - toda a pilha de aplicativos está operacional.
- Suporta vários ambientes:
  - desenvolvimento, preparação e produção.

Você deseja garantir que, ao implantar em uma velocidade maior, suas implantações não impactarão negativamente seus negócios/clientes. Existem diferentes mecanismos para auxiliar na validação e teste de código, como revisões por pares e automação. Com os microsserviços se tornando cada vez mais integrados às infraestruturas, é fundamental garantir que seu código possa ser integrado a essas dependências diretas.

Ações do AWS CodePipeline

Figura – Ações do AWS CodePipeline



Fonte: elaborado pelo autor

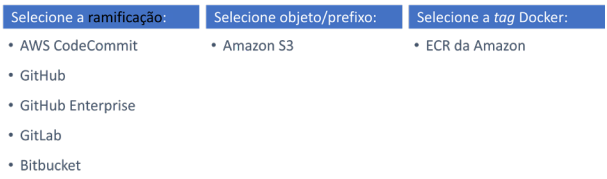
O AWS CodePipeline permite adicionar ações aos estágios do seu *pipeline* de CI/CD. Cada ação pode ser associada a um provedor que a executa. As categorias de ação do AWS CodePipeline e exemplos de provedores são os seguintes:

- **Fonte:** repositórios de código no AWS CodeCommit, Amazon S3, GitHub, Bitbucket, GitLab e Amazon ECR
- **Construir:** AWS CodeBuild, Jenkins
- Teste: Jenkins , BlazeMeter, Ghost Inspector
- **Implantar:** AWS Elastic Beanstalk e AWS CodeDeploy
- **Invocar:** AWS Lambda, AWS Step Functions, snyk
- **Aprovação:** Amazon SNS

Mais informações sobre integrações de produtos e serviços com o AWS CodePipeline (disponíveis em: http://docs.aws.amazon.com/codepipeline/latest/userguide/integrations-action-type.html).

AWS CodePipeline: fontes compatíveis

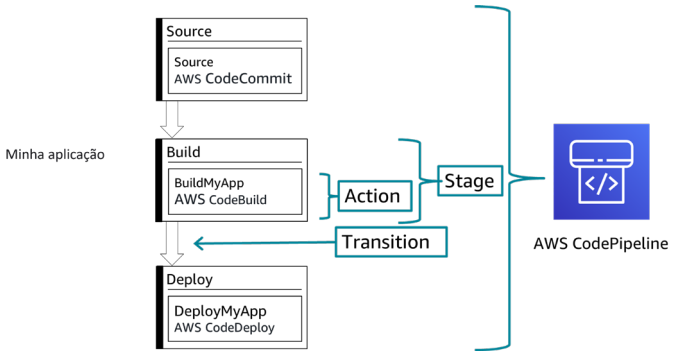
Enviar automaticamente uma versão e obter o código-fonte mais recente.



No lado do código-fonte, você pode selecionar sua ramificação no repositório de código-fonte, como AWS CodeCommit, GitHub e GitHub Enterprise e GitLab. Se estiver usando código do seu repositório local, também poderá armazená-lo no Amazon S3. Terceiro, selecione uma *tag* Docker (imagem base ou *tag* de lançamento da imagem base) para acionar uma compilação.

Um *pipeline* de três estágios

Figura – Um pipeline de três estágios



Fonte: elaborado pelo autor.

Este é um *pipeline* simples de três estágios. As ações de origem são ações especiais. Elas pesquisam continuamente os provedores de origem, como AWS CodeCommit , GitHub e Amazon S3, para detectar alterações. Quando

uma alteração é detectada, o novo *pipeline* é criado e inicia sua execução. As ações de origem recuperam uma cópia das informações de origem e a colocam em um *bucket* do Amazon S3 de propriedade do cliente.

Quando as ações de origem são concluídas, o estágio *Source* é marcado como bem-sucedido e o processo passa para o estágio *Build*.

O Build Stage contém uma ação: BuildMyApp. O AWS CodeBuild está integrado ao AWS CodePipeline como uma função nativa.

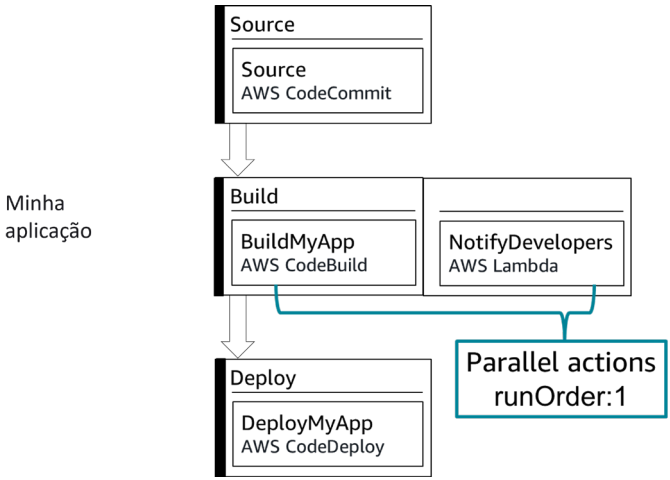
Você pode criar o projeto do AWS CodeBuild antes ou enquanto cria o pipeline do AWS CodePipeline.

Quando a ação de *build* for concluída, o estágio Build será marcado como bem-sucedido e o processo passará para o estágio Deploy.

O estágio Deploy contém uma ação: ação de implantação do AWS CodeDeploy. O AWS CodeDeploy action recupera o artefato de compilação do *bucket* do Amazon S3 de propriedade do cliente e o implanta no grupo de implantação do AWS CodeDeploy.

Ações paralelas em um *pipeline*

Figura – Ações paralelas em um pipeline



Fonte: elaborado pelo autor.

Existem alternativas para um *pipeline* simples. Neste exemplo, uma ação paralela para enviar uma notificação à equipe de desenvolvimento foi adicionada à fase de construção. Como BuildMyApp e NotifyDevelopers são ações paralelas, elas são iniciadas ao mesmo tempo.

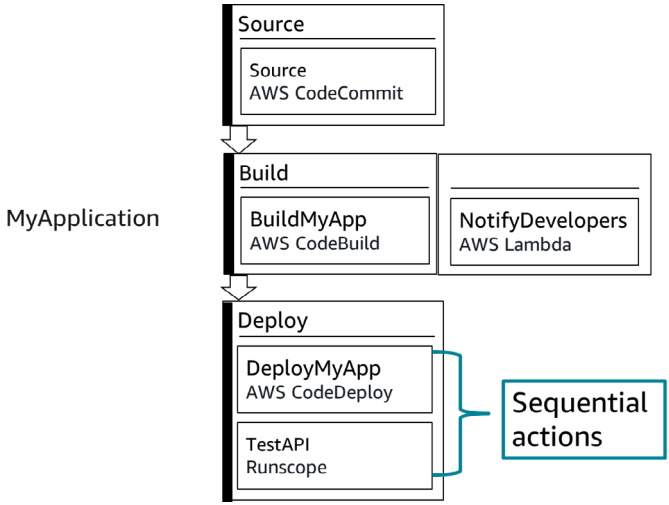
Em um estágio, você pode especificar a ordem em que as ações devem ser executadas, definindo o **runOrder da ação** como um valor inteiro. O valor runOrder padrão para uma ação é 1. O menor número indica a ação a ser executada

primeiro. Para especificar ações paralelas, use o mesmo número inteiro para cada ação que deseja executar em paralelo.

NOTA: por padrão, qualquer *pipeline* criado com sucesso no AWS CodePipeline tem uma estrutura válida. Você também pode criar ou editar manualmente um arquivo JSON para criar um ou atualizar um *pipeline* na AWS CLI.

Ações sequenciais em um *pipeline*

Figura – Ações sequenciais em um pipeline



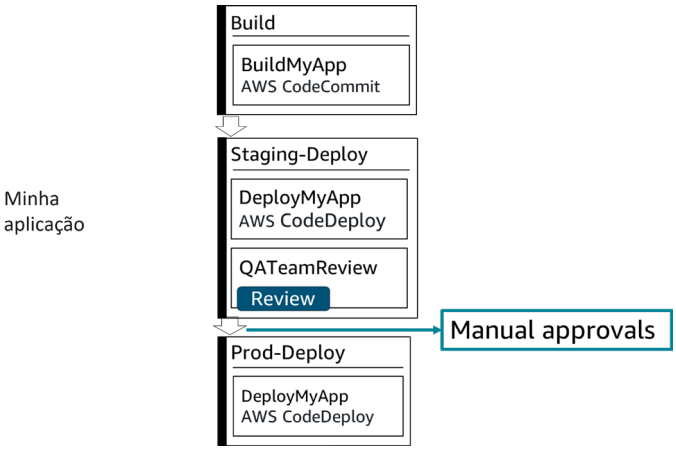
Fonte: elaborado pelo autor

Neste exemplo, uma ação sequencial foi adicionada ao estágio de implantação. A ação adicional, TextAPI, executa testes pós-compilação. A ação API de teste é iniciada somente após a ação DeployMyApp ser concluída com sucesso.

Cada estágio do *pipeline* pode conter ações paralelas e sequenciais.

Ações sequenciais em um *pipeline* – aprovações manuais

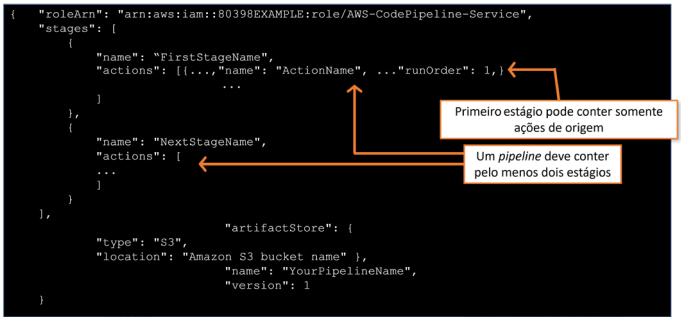
Figura – Ações sequenciais em um pipeline – aprovações manuais



Fonte: elaborado pelo autor.

Você pode adicionar uma aprovação manual no ponto em que deseja que o *pipeline* pare de funcionar até que alguém aprove ou rejeite a revisão em andamento. O *pipeline* para de ser executado quando atinge o ponto em que você definiu a ação de aprovação, retomando somente quando a ação for aprovada. A ação de aprovação é gerenciada com permissões do IAM. Você pode notificar os aprovadores de diversas maneiras, incluindo *e-mail*, AWS SMS, *webhooks* e muito mais.

Estruturando um *pipeline*



Ao criar um *pipeline* usando a AWS CLI, siga estas regras para sua estrutura:

- deve ter pelo menos duas etapas;
- o primeiro estágio deve conter uma ação de origem;

- somente o primeiro estágio pode ter uma ação de origem. Os estágios subsequentes atuam sobre a ação fonte;
- deve ter pelo menos um estágio diferente do estágio de ação de origem;
- Os nomes dos estágios no *pipeline* devem ser exclusivos.

Ao criar ações para um *pipeline* por meio da AWS CLI, aplicam-se as seguintes regras:

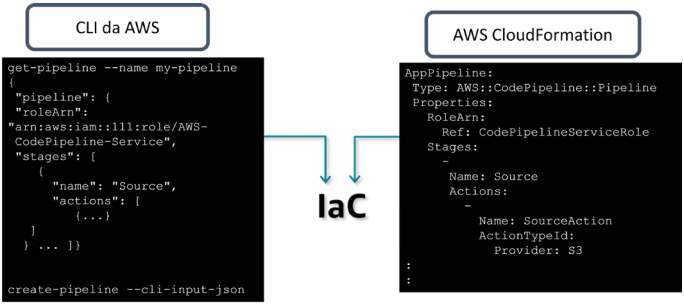
1. os nomes das ações devem ser exclusivos;
2. o nome do artefato de entrada de uma ação deve corresponder exatamente ao artefato de saída declarado em uma ação anterior;
3. use categorias de ação válidas:

Fonte, construção, aprovação, implantação, teste e invocação.

Estrutura de alto nível de uma ação

```
[ {
   "inputArtifacts": [ An input artifact structure, if supported for the action category ],
   "name": "ActionName",
   "region": "Region",
   "namespace": "source_namespace",
   "actionTypeId": {
          "category": "An action category",
          "owner": "AWS",
          "version": "1"
          "provider": "A provider type for the action category", },
   "outputArtifacts": [ An output artifact structure, if supported for the action category ],
   "configuration": { Configuration details appropriate to the provider type },
   "runOrder": A positive integer that indicates the run order within the stage,
} ]
```

Criação/atualização de um *pipeline* por meio de infraestrutura como código



Você pode criar e manter seus *pipelines* de várias maneiras:

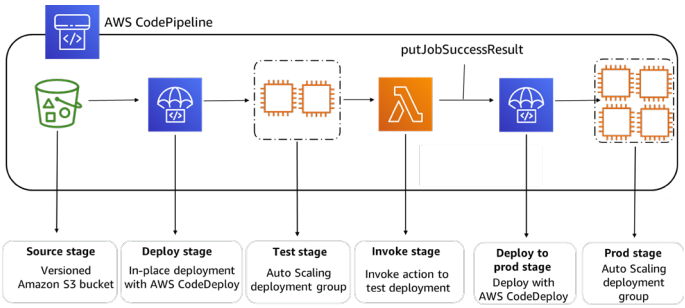
- use o AWS Management Console para criar *pipelines* facilmente;

- use a AWS CLI para exportar um *pipeline* no formato JSON (comando get-pipeline). Você pode posteriormente optar por atualizar o JSON e criar *pipelines* usando o arquivo JSON (comando create-pipeline);
- provisione *pipelines* usando o AWS CloudFormation.

Usando a AWS CLI com o *pipeline* formatado em JSON e o AWS CloudFormation, você pode gerenciar seus *pipelines* como código. Com essas opções, você obtém todos os benefícios do gerenciamento da infraestrutura como código.

Exemplo: invocar o AWS Lambda em um *pipeline*

Figura – Exemplo: invocar o AWS Lambda em um pipeline



Fonte: elaborado pelo autor.

Criptografia de artefato em um *pipeline*

Por padrão, o AWS CodePipeline usa criptografia no lado do servidor com as chaves de criptografia gerenciadas pelo Amazon S3 (SSE-S3) para proteger artefatos.

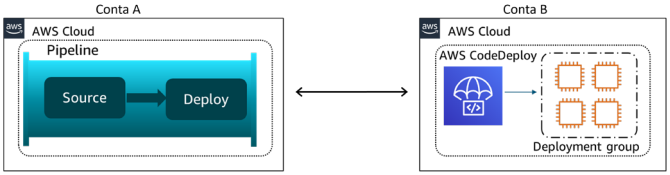
Melhores práticas:

- Use a chave gerenciada pelo cliente em vez da chave padrão do Amazon S3.
- Rotacione a chave de acordo com um cronograma para atender aos requisitos de segurança.

Usar recursos de outra conta da AWS em um *pipeline*

A conta A usa recursos do AWS CodeDeploy da conta. B

Figura – Usar recursos de outra conta da AWS em um pipeline



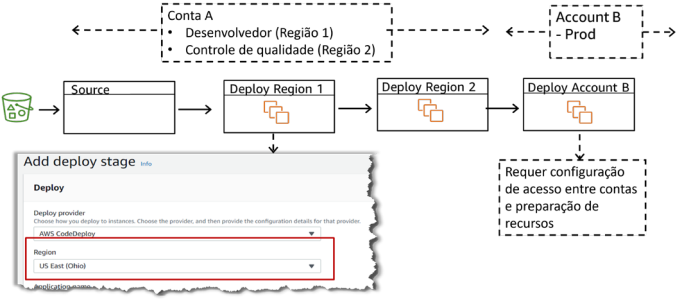
- Crie uma chave AWS KMS para usar no *pipeline*.
- Configurar políticas e funções da conta.
- Edite o *pipeline* para usar os recursos da Conta B.

- Configurar políticas de conta e função entre contas para permitir que a conta A acesse os recursos do AWS CodeDeploy da conta B.

Fonte: elaborado pelo autor.

Exemplo: Criando *pipeline* entre regiões/entre contas

Figura – Exemplo: Criando pipeline entre regiões/entre contas



Fonte: elaborado pelo autor

O exemplo do *slide* mostra como criar um único *pipeline* que é implantado em várias regiões na mesma conta e, em seguida, implantado em uma região pertencente a outra conta.

Nesse cenário, os *pipelines* de desenvolvimento e controle de qualidade estão em duas regiões diferentes da mesma conta da AWS (conta A), enquanto o *pipeline* de produção está em outra conta da AWS (conta B).

Para implantar em várias regiões da AWS na mesma conta da AWS, basta configurar o estágio de implantação para usar a região apropriada.

Para usar recursos criados ou gerenciados por outra conta da AWS, você deve configurar o acesso entre contas e configurar as ações do *pipeline* para que possam acessar artefatos nas limitações dos *pipelines* entre contas.

Para permitir o acesso entre contas, uma chave do AWS Key Management Service (AWS KMS) é adicionada ao *pipeline*, as políticas da conta são configuradas e as funções necessárias são criadas. A chave do AWS KMS é criada usando o ID da chave, o ARN da chave ou o ARN do alias.

As seguintes limitações gerais se aplicam a ações entre contas:

uma ação só pode consumir um artefato se alguma das seguintes situações se aplicar:

- a ação está na mesma conta que a conta do *pipeline*;
- o artefato foi criado na conta do *pipeline* para uma ação em outra conta;
- o artefato foi produzido por uma ação anterior na mesma conta da ação. Em outras palavras, você não poderá passar um artefato de uma conta para outra se nenhuma das contas for a conta do *pipeline*.

Para mais informações, veja:

"Crie um *pipeline* no CodePipeline que use recursos de outra conta AWS" (disponível em: https://docs.aws.amazon.com/codepipeline/latest/userguide/pipelines-create-cross-account.html).

Considerações ao projetar seu *pipeline* de CI/CD

- Estágios
- Tipos de testes
- Ordem dos testes
  - Qualquer um pode ser executado em paralelo?

- Detecte e relate falhas
- Regiões
- Provisionamento e gerenciamento
- Reversões

Considere estas questões ao projetar seu *pipeline* de CI/CD:

- Quais estágios você deseja em seu *pipeline*?
- Que tipos de testes você deseja executar em cada estágio?
- Quais testes devem ser executados sequencialmente?
- Quais testes podem ser executados em paralelo?
- Como o sistema detectará e reportará falhas?
- Em quais regiões da AWS seus serviços da AWS serão executados?
- Como você provisionará e gerenciará a infraestrutura para seus ambientes de teste e produção?
- Se forem detectados problemas, como você reverterá para uma versão anterior?

Considerações para entrega e implantação contínuas

Figura – Considerações para entrega e implantação contínuas

Antipadrão	Padrão
Intervenção manual	Processo automatizado
Codificando valores de configuração no código-fonte ou usando ferramentas diferentes em cada ambiente de desenvolvimento	Todas as configurações valores e ferramentas externalizados em propriedades e recursos de tempo de construção/implantação
Apenas executando análises/testes no estágio de *commit*, levando muito tempo para descobrir problemas	Fazer com que o *commit build* seja executado o mais rápido possível para descobrir problemas comuns de *build*
Fazendo alterações de última hora para forçar uma compilação a funcionar	Falhar rapidamente usando CD reduz a necessidade de correções de última hora
Escrita de um aplicativo em que cada componente está fortemente acoplado a todos os outros	Desacoplando componentes do seu aplicativo para permitir atualizações de componentes individuais

Fonte: elaborado pelo autor.

Aqui está um exemplo de alguns antipadrões comuns de entrega e implantação e os padrões corrigidos que um sistema contínuo oferece.

Ter um sistema que requer intervenção manual quando ela pode ser evitada retarda o seu processo. Portanto, usar a automação para CD acelera seu processo.

Um antipadrão comum é manter os valores de configuração de uma construção no próprio código ou até mesmo ter desenvolvedores usando ferramentas diferentes em seus processos de construção. Isso leva a compilações inconsistentes entre os desenvolvedores e a

uma perda geral de tempo e esforço tentando conciliar por que algumas compilações funcionam em alguns ambientes e não em outros. A externalização dessas configurações e ferramentas as mantém consistentes entre as compilações, permite melhor automação e que seu processo seja dimensionado com mais facilidade.

Não usar um processo de CD pode levar a pressas de última hora no meio da noite para fazer uma compilação funcionar. Projete seu processo de CD para “falhar rapidamente” para reduzir a probabilidade de surpresas de última hora.

5

AWS CodeBuild

Serviço: AWS CodeBuild



AWS CodeBuild

Um serviço de *build* totalmente gerenciado que compila código-fonte, executa testes e produz pacotes de *software*.

Benefícios:

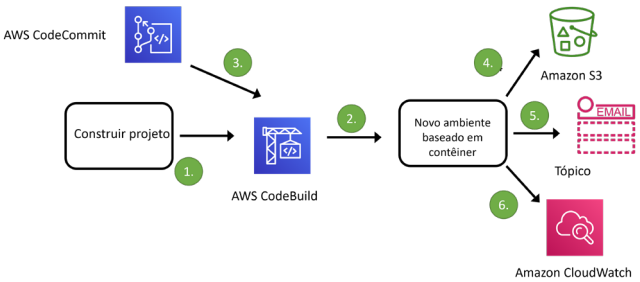
- totalmente gerenciado;
- escala continuamente;
- fornece monitoramento;
- extensível;
- acessível por meio do console AWS e da CLI.

Os benefícios do AWS CodeBuild incluem:

- serviço de *build* totalmente gerenciado, portanto não há servidores de *build* para configurar, atualizar e instalar *software* e gerenciar por conta própria;
- escala continuamente para atender ao seu volume de construção. Ele pode executar várias compilações simultaneamente;
- monitora compilações por meio do CloudWatch Events.
- CodeBuild vem com ferramentas de compilação e tempos de execução predefinidos. No entanto, o CodeBuild é extensível e você pode criar seus próprios ambientes personalizados com suas próprias ferramentas de compilação e tempos de execução.

Como funciona o AWS CodeBuild

Figura – Como funciona o AWS CodeBuild



Fonte: elaborado pelo autor.

1. Forneça ao AWS CodeBuild um projeto de compilação. Um arquivo de projeto de *build* contém informações sobre onde obter o código-fonte, o ambiente de *build* e como compilar o código. O componente mais importante é o arquivo buildspec.

2. AWS CodeBuild cria o ambiente de compilação. Um ambiente de construção é uma combinação de sistema operacional, tempo de execução da linguagem de programação e outras ferramentas necessárias para a construção.

3. O AWS CodeBuild faz *download* do código-fonte no ambiente de compilação e usa o arquivo buildspec para executar uma

compilação. Esse código pode ser de qualquer provedor de origem; por exemplo: repositório GitHub, Amazon Bucket de entrada S3, repositório Bitbucket ou repositório AWS CodeCommit.

4. Os artefatos de compilação produzidos são carregados em um *bucket* do Amazon S3.

5. O ambiente de construção envia uma notificação sobre o *status* da construção.

6. Enquanto a compilação está em execução, o ambiente de compilação envia informações para o Amazon CloudWatch Logs.

Ferramentas adicionais para criar ambientes









Guias de início rápido da AWS

A seguir, são listadas algumas das muitas ferramentas diferentes e *software* de construção disponíveis.

Bambu

Bamboo é uma ferramenta popular que "une construções, testes e lançamentos automatizados em um único fluxo de trabalho". Use o Bamboo para integrar e usar outros *softwares* Jira. Algumas outras características exclusivas do Bamboo Server são os fluxos de trabalho ramificados do Git, a capacidade de automação de testes (sem um *plugin* adicional) e a integração do *software* Jira.

(Fonte: https://www.atlassian.com/software/bamboo).

Jenkins

Uma das principais características que o Jenkins oferece é uma instalação simples. Também promove configurações fáceis. Jenkins é uma ferramenta de código aberto com um repositório de ferramentas que você pode usar.

(Fonte: https://wiki.jenkins.io/display/JENKINS/Meet+Jenkins).

EquipeCidade

Uma das principais características do TeamCity é a capacidade de executar e verificar testes automatizados no servidor antes de um *commit*. Além disso, há uma barra de progresso de construção que informa instantaneamente quando algo está errado.

(Fonte: https://www.jetbrains.com/teamcity/).

Guias de início rápido da AWS

Os *Quick Starts* são criados por arquitetos de soluções e parceiros da AWS para ajudar você a implantar tecnologias populares na AWS, com base nas práticas recomendadas da AWS para segurança e alta disponibilidade. Esses aceleradores reduzem centenas de procedimentos manuais em apenas algumas etapas, para que você possa criar seu ambiente de produção rapidamente e começar a usá-lo imediatamente.

Cada *Quick Start* inclui modelos do AWS CloudFormation que automatizam a implantação e um guia que discute a arquitetura e fornece instruções passo a passo de implantação.

Como planejar uma construção?

- Identifique onde o código-fonte está localizado.
- Identifique os comandos de compilação que você precisa executar e a ordem em que quer executá-los.
- Identifique tempos de execução e ferramentas.
- Identifique os recursos se não forem fornecidos automaticamente.

Identifique onde o código-fonte está localizado:

- **AWS CodeCommit /GitHub:** forneça o nome do repositório;
- **Amazon S3:** forneça o nome do *bucket;*
- **Amazon Elastic Container Registry (Amazon ECR):** etiqueta Docker.

Identifique os comandos de compilação que você precisa executar e a ordem em que deve executá-los.

- **Use o balão de fala:** codeINTRO: para instruir como transformar a entrada de compilação baixada na saída de compilação esperada.

Identifique tempos de execução e ferramentas

- **Linguagem de programação:** Java, Ruby, Python ou Node.js.
- **Ferramentas de construção:** Maven, Ant ou um compilador para Java, Ruby ou Python.

Identifique os recursos, se não forem fornecidos automaticamente pelo AWS CodeBuild.

Você precisa de recursos da AWS que não são fornecidos automaticamente pelo CodeBuild? Em caso afirmativo, de quais políticas de segurança esses recursos necessitarão? Por exemplo, talvez seja necessário modificar a função de serviço do CodeBuild para permitir que o CodeBuild funcione com esses recursos.

Além disso, o AWS CodeBuild executa compilações em ambientes de compilação que usam imagens Docker. Essas imagens do Docker devem ser armazenadas em um tipo de repositório compatível com o AWS CodeBuild, como Docker Hub e Amazon ECR.

Depois de identificar os componentes necessários, você terá as configurações e

os recursos necessários para executar uma compilação com êxito. Para executar sua compilação, você pode usar o console do AWS CodeBuild, a AWS CLI, os SDKs da AWS ou a API HTTP do AWS CodeBuild.

Melhores práticas para tratamento de dependências

- Habilite o cache do AWS CodeBuild para armazenar dependências em *cache*.
- Usar a política de ciclo de vida do Amazon S3.
- Substitua o comportamento do *cache* atualizando o projeto.
- Habilite o *cache* para qualquer pasta no ambient.e de construção.

Um processo típico de construção de aplicativo inclui fases, como:

- preparando o ambiente;
- atualizando a configuração;
- baixando dependências;
- executando testes unitários;
- empacotando o artefato construído.

Baixar dependências é uma fase crítica no processo de construção. Esses arquivos dependentes podem variar em tamanho de alguns KB a vários MB. Como a maioria dos arquivos dependentes não mudam frequentemente entre compilações, você pode reduzir significativamente o tempo de compilação armazenando dependências em *cache*.

Por padrão, o arquivo em *cache* é criptografado no lado do servidor com a chave AWS KMS do artefato do cliente.

Melhores práticas para *cache*

- Expire o *cache* removendo manualmente o arquivo de *cache* do Amazon S3. Como alternativa, você pode expirar o *cache* usando uma política de ciclo de vida do Amazon S3.
- Substitua o comportamento do *cache* atualizando o projeto. Você pode usar o console do AWS CodeBuild, a AWS CLI ou os AWS SDKs para atualizar o projeto. Você também pode invalidar as configurações de *cache* usando a nova

API InvalidateProjectCache. Essa API força a geração de uma nova InvalidationKey, garantindo que compilações futuras recebam um *cache* vazio. Essa API não remove o *cache* existente, pois isso pode causar inconsistências na execução de compilações.

- Habilite o cache para qualquer pasta no ambiente de construção; no entanto, armazene em *cache* apenas dependências/arquivos que não serão alterados com frequência entre compilações. Além disso, para evitar comportamento inesperado do aplicativo, não armazene em *cache* informações confidenciais e de configuração.

Mais informações sobre como habilitar o *cache* para AWS CodeBuild (disponíveis em: https://aws.amazon.com/blogs/devops/how-to-enable-caching-for-aws-codebuild/).

Construir projeto para AWS CodeBuild



1. Execute o aws codebuild create-project -- comando generate-cli-skeleton. Isso cria uma saída JSON com a estrutura do projeto de construção.

2. Modifique o esqueleto gerado com o seguinte:

   - nome do projeto do código-fonte;
   - localização do código-fonte;
   - tipo e localização do arquivo de artefatos de saída;
   - ambiente de construção;

do IAM que permite que o AWS CodeBuild acesse os recursos

Chave de criptografia para criptografar os artefatos de saída do build

Nota : Neste exemplo, o arquivo buildspec não foi mencionado nos atributos "fonte". Isso significa que o arquivo ZIP do código-fonte contém o arquivo buildspec. Embora o atributo buildspec seja opcional, é importante especificá-lo no arquivo do projeto de construção ou incluí-lo no arquivo ZIP do código-fonte.

3. Execute o aws codebuild create-project -- arquivo cli-input-json : *//create-project.json*.

O create- project.json é o nome do arquivo do projeto que você modificou. O comando retorna uma saída formatada em JSON do projeto de construção criado.

Você pode visualizar o projeto de compilação no Console de gerenciamento da AWS na lista Projetos de serviço do AWS CodeBuild.

Observação: o AWS CodeBuild também possui uma opção orientada por UI para criar o projeto de construção.

Mais informações (disponíveis em: http://docs.aws.amazon.com/codebuild/latest/userguide/create-project.html).

Fases de buildspec

Figura – Fases de buildspec

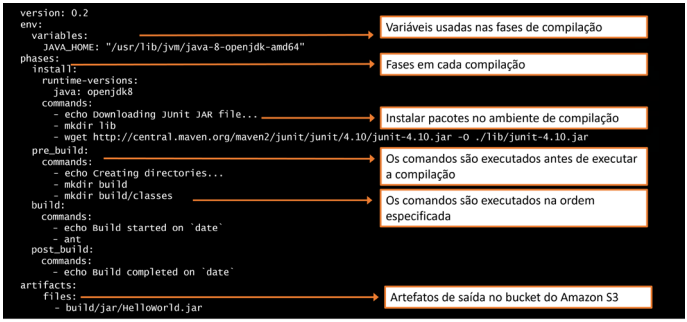
Estágio	Descrição	Exemplos
instalar	Instalação de pacotes no ambiente	Instale estruturas de teste. Por exemplo, RSpec, Mocha, Java, Android, Ruby
pré_construção	Comandos a serem executados antes da compilação, como etapas de login ou instalação de dependências	Faça login no Amazon ECR. Execute o empacotador Ruby ou npm
construir	Sequência para executar a compilação, como compilação, execução de testes ou ambos	Execute qualquer comando de construção como "go build.“ Execute comandos de teste compatíveis com Mocha, RSpec ou sbt
pós_construção	Comandos a serem executados após uma compilação com sucesso ou falha	Maven para empacotar os artefatos de compilação em um JAR/WAR ou enviar uma imagem Docker para o Amazon ECR

Fonte: elaborado pelo autor.

Esta tabela analisa rapidamente as diferentes fases do arquivo de especificação de construção. O Docker também pode usar o AWS CodeBuild para criar e testar seu código (mais detalhes no módulo de serviços de contêiner).

Mais informações sobre referências de buildspec (disponíveis em: https://docs.aws.amazon.com/codebuild/latest/userguide/build -spec-ref.html#build-spec-ref-name-storage).

Criando um especificação de construção arquivo YAML



O arquivo buildspec é um arquivo YAML com a estrutura e sintaxe mostradas. Inclui uma coleção de comandos de compilação e configurações relacionadas que o AWS CodeBuild usa para executar uma compilação. Você pode incluir o arquivo buildspec junto com o código-fonte ou defini-lo no projeto de construção.

Se você incluir o arquivo buildspec com o código-fonte, nomeie -o como buildspec.yml.

No buildspec versão 0.1, o AWS CodeBuild executa cada comando shell em uma instância separada no ambiente de compilação. Isso significa que cada comando é executado isoladamente de todos os outros comandos. Portanto , por padrão, você não pode executar

um único comando que dependa do estado de qualquer comando anterior (por exemplo, alterar diretórios ou definir variáveis de ambiente). Para contornar essa limitação, use a versão 0.2, que resolve esse problema.

Se você precisar usar a versão 0.1, a AWS recomenda as seguintes abordagens:

- Inclua um script de shell em seu código-fonte que contenha os comandos que você deseja executar em uma única instância do shell padrão. Por exemplo, você pode incluir um arquivo chamado my-script.sh em seu código-fonte que contém comandos como cd MyDir ; mkdir -p meuSubDir ; cd meuSubDir ; senha ; Em seguida, no arquivo buildspec , especifique o comando./my-script.sh.
- No arquivo buildspec ou na configuração Comandos de construção somente para a fase de construção, insira um único comando que inclua todos os comandos que você deseja executar em uma única instância do shell padrão (por exemplo), cd MyDir && mkdir -p meuSubDir && cd meuSubDir && senha

Exemplo: Criando um buildspec do Docker



Exemplo : Docker para AWS CodeBuild

Este código de exemplo produz uma imagem do Docker como saída de compilação e, em seguida, envia a imagem do Docker para um repositório de imagens do Amazon ECR. Você também

Neste exemplo, observe as fases de instalação, pré-construção, construção e pós-construção. A pós-construção exibe o envio da imagem do Docker para o Amazon ECR. Você também pode enviar para um Docker Hub.

Para obter mais informações sobre como enviar para um Docker Hub, consulte https://docs.aws.amazon.com/codebuild/latest/userguide/sample-docker.html.

Protegendo suas construções

- Crie permissões para acessar recursos de build
- Políticas gerenciadas pela AWS específicas para AWS CodeBuild
  - AWSCodeBuildAdminAccess
  - AWSCodeBuildDeveloperAccess
  - AWSCodeBuildReadOnlyAccess
  - AmazonS3ReadOnlyAccess

Crie permissões para acessar recursos de *build*

- Para armazenar e recuperar artefatos de *build* em *buckets* do Amazon S3.
- Para visualizar o Amazon CloudWatch Logs para compilações.
- Para criar, visualizar ou excluir projetos de *build* e para iniciar e interromper *builds*.

Políticas gerenciadas pela AWS específicas para AWS CodeBuild

- AWSCodeBuildAdminAccess: fornece acesso total ao AWS CodeBuild, incluindo

permissões para administrar projetos de construção do AWS CodeBuild.

- AWSCodeBuildDeveloperAccess: fornece acesso ao AWS CodeBuild, mas não permite construir administração do projeto.
- AWSCodeBuildReadOnlyAccess: fornece acesso somente leitura ao AWS CodeBuild.
- AmazonS3ReadOnlyAccess: fornece acesso para criar artefatos de saída criados pelo AWS CodeBuild.

Controle o acesso ao AWS CodeBuild usando funções do IAM

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "codebuild:CreateProject",
      "Resource": "arn:aws:codebuild:us-east-1:123456789012:project/*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam:123456789012:role/CodeBuildServiceRole"
    }
  ]
}
```

Este exemplo de declaração de política permite que um usuário crie projetos de *build* na região us-east-1 para a conta 123456789012. Anexar este exemplo de política a um usuário

permite que ele crie projetos de *build* na conta e região especificadas e passe a política " CodeBuildServiceRole " para CodeBuild para que possa executar ações em seu nome. Isso restringe a atribuição do CodeBuildServiceRole apenas aos usuários que têm a capacidade de passar para ele.

Mais informações sobre as permissões necessárias ao CodeBuild (disponíveis em: https://docs.aws.amazon.com/codebuild/latest/userguide/auth-and-access-control-iam-identity-based-access-control.html).

Integrações de produtos AWS CodeBuild



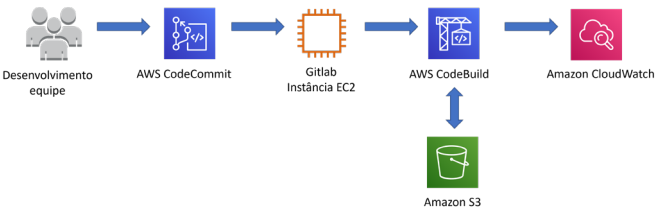
O AWS CodeBuild pode se conectar ao AWS CodeCommit, Amazon S3, GitHub Enterprise e Bitbucket para extrair código-fonte para compilações. Com os recursos colaborativos do GitHub.com, indivíduos e equipes podem escrever códigos melhores e mais rápido.

Bitbucket é uma ferramenta de gerenciamento de repositório Git para equipes. Colabore no código-fonte, proteja seu fluxo de trabalho com permissões e verificações e crie um pipeline de lançamento.

Para remover o trabalho pesado de gerenciar seu farm de build Jenkins, a AWS desenvolveu um plug-in Jenkins AWS CodeBuild. Após a ativação do plug-in, um desenvolvedor pode configurar um projeto Jenkins para obter novos commits do repositório de código-fonte escolhido e executar automaticamente as compilações associadas. Falaremos mais sobre isso em um momento.

Exemplo: Processo de lançamento de desenvolvimento

Figura – Exemplo: Processo de lançamento de desenvolvimento



Fonte: elaborado pelo autor.

O exemplo a seguir usa AWS CodeCommit (Git) como gerenciamento de controle de origem

(SCM) e Amazon S3 para armazenamento de artefatos de construção. Os *logs* são armazenados no CloudWatch. Um *pipeline* de desenvolvimento que usa Jenkins com arquitetura de *plug-in* CodeBuild se parece com este diagrama.

Para tornar a integração com Jenkins o mais simples possível, você pode baixar o modelo AWS CloudFormation (disponível em: https://s3.amazonaws.com/proberts-public/jenkins.yaml).

Faça *download* do modelo, faça *login* no console do AWS CloudFormation e use o modelo para criar uma pilha.

Mais informações sobre uma compilação do Jenkins com AWS CodeBuild (disponíveis em: https://docs.aws.amazon.com/codebuild/latest/userguide/jenkins-plugin.html).